



ДИПЛОМЕН ПРОЕКТ

НА ТЕМА

„Разработка на уеб-базиран форум за футболни ентусиасти с интегрирани социални и игрови функционалности - FutBrain“

Ученик:

Георги Здравков Примов

Професия: Системен програмист

Специалност: Системно програмиране

Ръководител консултант:

Росен Димитров Вълчев

гр. Пловдив, 2026

Съдържание



Съдържание

I.	Увод	6
II.	Цел на проекта	7
1.	Висока интерактивност	7
2.	Модерен и лесен за разбиране графичен потребителски интерфейс	7
3.	Мултиплатформена съвместимост	8
4.	Поддържане на актуален игрови модул	8
5.	Стимулиране на добрите отношения между потребителите чрез човешка и AI модерация	8
III.	Проучване на съществуващи приложения	9
IV.	Обхват на проекта	11
1.	Потребителска част	11
2.	Администраторска част	11
3.	Ограничения на системата	11
V.	Използвани технологии	12
1.	Visual Studio Code	12
2.	Език за програмиране – TypeScript	13
3.	Backend	13
3.1	Node.js + Express	13
3.2	База от данни – MongoDB Atlas, Mongoose и Typegoose	14
3.3.	Работа с базата от данни - Mongoose и Typegoose	14
3.4.	Сигурност - bcrypt	15
3.5.	Авторизация - JSON Web Tokens (JWT)	15
3.6.	Обработка на бисквитките - cookie-parser	15
3.7.	Управление на сървърните пътища - CORS	16
3.8.	Качване и съхранение на файлове - Multer	16
3.9.	Управление на изходяща комуникация чрез ел.поща – Nodemailer	16
3.10.	Внедряване на функции с ИИ за генериране на съдържание - google/generative-ai/gemini-2.5-flash-lite	17
3.11.	Конфигурация на средата - dotenv	17
4.	Frontend	17
4.1.	Библиотека за потребителски интерфейс - React	18
4.2.	Инструмент за изграждане и разработка - Vite	18
4.3.	Стилизиране на компонентите и адаптивен дизайн - CSS	18
4.4.	Икони за модерен изглед - Font Awesome	19



5. Лого на FutBrain – Adobe Photoshop.....	19
VI. Системна архитектура	20
1. Backend архитектура (src/) (фиг.3)	20
1.1. Models (models/)	20
1.2. Repositories (repositories/)	20
1.3. Services (services/)	20
1.4. Routes (routes/)	20
1.5. Middlewares (middlewares/)	20
1.6. DTO-та (dtos/)	20
1.7. Node Modules (node_modules/)	21
1.8. Distribution (dist/)	21
1.9. Uploads (uploads/)	21
1.10. App (/app.ts)	21
2. Frontend архитектура (frontend/)	21
2.1. Public (public/)	21
2.2. Pages (src/pages/)	22
2.3. Layouts (src/layouts/)	22
2.4. Styles (src/styles)	22
2.4. Auth (src/auth/)	22
2.5. DTO-та (src/dtos/)	22
2.6. App (src/App.tsx)	22
2.7. Main (src/main.tsx)	23
2.8. Node modules (/node_modules)	23
VI. Описание на системата	24
1. Архитектура на базата от данни (фиг. 5)	24
1.1. User (Потребител)	24
1.2. Accessory (Аксесоар)	24
1.3. Comment (Коментар)	24
1.4. Post (Публикация)	24
1.5. Reply (Отговор на коментар)	24
1.6. Player (Футболен играч)	24
1.7. Game (Игра)	25
1.8. Report (Доклад)	25
2. Алгоритми	26
2.1. Алгоритъм за сигурност и криптографска защита	26



2.2. Алгоритъм за автоматизирана AI модерация.....	26
2.3. Алгоритъм за управление на виртуалната икономика (Futcoins).....	26
2.4. Логика за динамично управление на ресурси	26
3. Документация на кода.....	27
3.1. Декоратори и Hooks (@pre "save")	27
3.2. Клас за централизирано управление на грешки (AppError).....	27
3.3. Middleware за JWT автентикация	28
3.4. Метод comparePasswords	28
VII. Ръководство за потребителя.....	29
1. Инструкции за работа със системата.....	29
1.1. Регистрация и сигурност на достъпа.....	29
1.2. Навигация и потребителски интерфейс	29
1.3. Социална интеракция и модерация	30
1.4. Магазин за аксесоари и инвентар:.....	30
1.5. Игрови модул (Game)	31
VIII. Авторски права	32
IX. Заключение	33
X. Използвана литература	35



Списък с фигури

Фигура 1: Работа с VS Code	12
Фигура 2: Изработка на логото в Adobe Photoshop.....	19
Фигура 3: Архитектура на фронтенда.....	23
Фигура 4: Архитектура на бекенда.....	23
Фигура 5: Архитектура на базата от данни	25
Фигура 6: @pre "save" кука на обекта User	27
Фигура 7: Хвърляне на AppError при автентикация.....	28
Фигура 8: След гласуване в "Най-добър играч на седмицата"	30
Фигура 9: Преглед на публикация с коментари и отговори	30
Фигура 10: Изглед на магазина на FutBrain.....	31



I. Увод

С развитието на социалните мрежи всички търсят съмишленици, за да могат заедно да обсъждат любопитните събития около любимите си занимания. На практика всеки в съвременния свят има достъп до устройство, с което да комуникира със събеседник независимо от разстоянието помежду им. Вследствие на модернизацията и глобализацията става все по-осезаема нуждата хората да изразяват своето мнение по вълнуващите ги теми. Масово сред най-любимите теми е от спортната индустрия и тя се нарича футбол. Футболът е най-популярният спорт в света и хората търсят всевъзможни начини да се забавляват с него. В последните две десетилетия кривата на тактическото развитие на спорта бележи рязък скок и всяко едно качество на играчите се оценява от журналисти и публика. За разлика от миналия век талантът вече не е толкова ключов фактор, а се извлича максимумът от всяко едно качество на всеки един спортист. Дори всяка една малка неточност, всеки един малък момент на гениалност могат да бъдат решаващи за изхода от даден футболен мач. Каквото става на тренировъчния терен, намира отражение в двубоите, а тактическите интерпретации са за публиката.

За да може всеки да изрази мнението си по отношение на така развития днес футбол, е необходима специализирана за тази цел социална платформа на футболна тематика. Основните функции на един подобен форум, за да се поддържа интересът на аудиторията, която иска да изрази своята гледна точка по въпросите на деня, и да се стимулира конкуренцията между потребителите в платформата, трябва да бъдат:

- Предоставяне на интуитивен и лесен за разбиране графичен потребителски интерфейс
- Добавяне на механизми, чрез които потребителите да изразяват мнението си – публикации, коментари и отговори на коментари
- Поддържане на игрови модул, който дава възможност на потребителя да заяви своите лични предпочитания

Футболният форум „FutBrain” ще бъде разработен като една качествена уебплатформа, която съчетава и трите базови функционалности, за да предостави на потребителя едно запомнящо се преживяване, събиращо на едно място забавлението и реализиращо в пълна степен свободата за изразяване на лична позиция.



II. Цел на проекта

Безплатната уеб-базирана платформа „FutBrain” е предназначена за футболни ентусиасти, които желаят да съчетаят любимия си спорт с обсъждането на вълнуващите ги теми, свързани с него, заедно с други фенове. Уебсайтът дава възможност за приятно и забавно преживяване в Интернет съвместно с други запаянковци, изпълнено с интересни факти, гледни точки, спорове и игра, служеща за надпреварване между потребителите чрез прилагане на футболен интелект или следене на тенденциите по отношение на всеобщото мнение към играчите. Но също така ще има администрация и внедрени технологии, които ще се грижат за ползотворното и лишено от агресивни онлайн конфликти взаимодействие между потребителите. Достъпът до всички потребителски функции на форма е възможен само с една лесна регистрация. За да бъде разработена платформата да удовлетворява интересите и желанията на потенциалните си потребители, тя ще разполага със следните функционалности за един качествен уебсайт:

1. Висока интерактивност

Уебплатформата разчита на CSR (Client-Side Rendering) - зареждане на съдържанието от страна на клиента (браузъра). При този подход клиентът (браузъра на потребителя) взима готовия дизайн на HTML страницата и тя се показва на екрана, а за останалата необходима информация се изпълнява код, който я извлича от Backend-а. Получената информация се извежда на екрана след получаването ѝ. Така при всяка промяна на базата от данни се актуализира съдържанието на страницата автоматично, без да се презарежда. Този начин на работа наподобява Desktop приложение. В сравнение с евентуален SSR подход (Server-Side Rendering) - зареждане на страниците от сървър, натовареността върху Backend-а е по-малка и той извършва по-малко работа, за да обслужи повече потребители. CSR рамките са удобни за интерактивни приложения като FutBrain, в които навигацията не изисква пълно презареждане на страницата, а се актуализира само тази част, при която е имало промяна – колекции, идентификатори и съдържание.

2. Модерен и лесен за разбиране графичен потребителски интерфейс

Интерфейсът на FutBrain е разработен така, че да бъде максимално бързо разбираем от всеки един посетител на сайта независимо от неговите компютърни умения. Изгледът ще има ясна и интуитивна логическа структура, за да се постигне най-



бърза ориентация в уебсайта и да се предостави най-доброто изживяване за потребителя. Визуалните елементи на уебстраниците като бутони и линкове често са съпроводени с иконки, за да се избегне сухотата им. Основните функции на уебсайта са разположени на открити и удобни места за потребителите, като така се позволява максимално бързо изпълнение на задачи без излишни стъпки. Дизайнът на уебсайта се състои основно от приятни за човешкото око цветове – синьо, зелено и бяло. Синьото има положителен ефект върху когнитивното здраве на организма, изразяващ се в психологическо успокоение и стимулиране на креативността. Зеленото, от една страна, е цветът на тревата по футболните игрища, като наемква за тематиката на платформата, а от друга, изгледът към зелени пространства подобрява настроението и намалява стреса. Белият цвят се свързва с простотата, чистотата и вътрешния мир.

3. Мултиплатформена съвместимост

FutBrain, в качеството си на уебплатформа, ще бъде достъпна за абсолютно всички операционни системи, стига на тях да бъдат достъпни необходимите средства за достъп до Интернет – браузърите. Тя ще бъде съвместима с всички модерни браузъри – Google Chrome, Firefox, Brave, Safari, и ще ѝ бъде осигурен адаптивен интуитивен дизайн в зависимост от устройството на клиента, за да е достъпна навсякъде и по всяко време.

4. Поддържане на актуален игрови модул

Игровият модул във FutBrain предизвиква състезателния дух на потребителите и им дава възможност да изразят своята футболна експертиза или наблюденията си по отношение на глобалните виждания относно футболистите. Паралелно с целта на платформата за обсъждане на актуални теми във футбола ще бъдат актуализирани и играчите от страна на администраторите в случай на смяна на отбор. Потребителите ще бъдат награждавани при успех в играта, а наградата те ще могат да използват за свой стимул за бъдещо развитие и успех.

5. Стимулиране на добрите отношения между потребителите чрез човешка и AI модерация

В социална среда като FutBrain взаимодействието между потребителите може да доведе до агресивно налагане на мнение чрез нецензурни, расистки, дискриминационни или политически реплики. При съобщения с по-видимо подобно съдържание ще има наличие на изкуствен интелект, който да предупреждава потребителите да променят написаното от тях, но ако съобщението има скрит подтекст според потърпевшия, той ще има правото да изпрати доклад до администрацията.

III. Проучване на съществуващи приложения

В момента съществуват многобройни решения на футболните желания на потребителите, но липсва централизирана платформа с лесен за разбиране и съвременен графичен потребителски интерфейс, която да обединява на едно място хора с интерес към този спорт. Настоящият анализ разглежда подбрани решения, които представят общото състояние на проектите със сходна насоченост като FutBrain, като се акцентира върху техните основни характеристики и ограничения.

Едно от решенията, наподобяващо целта и предназначението на FutBrain, са групите и страниците за футбол във Facebook. Те позволяват обсъждане и споделяне на съдържание, но имат съществени недостатъци. Първо, публикациите не достигат до всички потребители, тъй като видимостта им зависи от алгоритъм. Това произтича от широкия обхват на платформата, която обединява аудитория с разнообразни интереси. Второ, структурата на Facebook групите често е неорганизирана поради големия обем от съдържание. Трудно се откриват стари дискусии, а отделните теми се развиват с времето. Освен това липсват специализирани механизми за ангажиране на потребителите и ясно дефинирани методи за модерация на специфично съдържание.

Друго известно, но частично решение на проблема с футболните дискусии, е уебплатформата Reddit. В нея съществуват популярни форуми като r/soccer и r/football, които предлагат категоризирани публикации, коментари и система за оценяване чрез гласуване. Въпреки това липсват персонализирани профили с натрупване на точки и елементи на геймификация, които да стимулират по-активно участие. Допълнителен недостатък е широкият обхват на самата платформа, която не е насочена изцяло към футболна аудитория. Това затруднява ефективното търсене и подреждане на съдържание по конкретни теми.

Най-близки до предназначението на FutBrain са специализираните спортни портали като Sports.ru и Goal.com. Те предоставят ежедневни новини от света на футбола с възможност за коментиране от регистрирани потребители. Въпреки това отсъства по-широка социална интеграция и липсват функционалности, свързани с игрови елементи. Интерфейсът на значителна част от тези сайтове не отговаря на съвременните изисквания за удобство, което затруднява ориентацията на потребителите. Съществуват и платформи като premierleague.com и championsleague.com, които включват модули за



съревнование между потребителите, но не предоставят достатъчно възможности за свободно изразяване на мнение и участие в дискусии.

Въз основа на направеното проучване може да се заключи, че съществуващите решения са частично ефективни за комуникация и обмен на информация, но не предлагат цялостно съчетание от социални функции, игрови механизми и целенасочена модерация чрез изкуствен интелект. Често се наблюдава и остарял визуален облик, който затруднява навигацията. Налице е необходимост от специализирана платформа, която да обединява съдържание, комуникация и социално взаимодействие в единна среда. FutBrain има за цел да централизира футболната общност на едно място, да интегрира липсващите функционални елементи и да предостави структурирано и последователно потребителско изживяване, съобразено с нуждите на футболната общност.



IV. Обхват на проекта

1. Потребителска част

- Автентикация и профил: Регистрация, вход, управление на потребителски профил и разглеждане на активност (публикации, коментари).
- Съдържание: Добавяне, разглеждане, харесване и сортиране на публикации по категории (отбори, играчи, ключови думи при търсене).
- Дискусии: Добавяне на коментари под публикациите и отговори (replies) към тях.
- Интерактивност: Участие в седмична игра за избор на "Най-добър играч", натрупване на виртуална валута (futcoins) и закупуване на дигитални аксесоари.
- Социални елементи: Разглеждане на класации (по futcoins, харесвания и публ).
- Обратна връзка: Докладване на неуместно съдържание и свързване с администрацията чрез страница "Контакти".

2. Администраторска част

- Управление на съдържанието: Пълен достъп до базата данни, възможност за изтриване на публикации, коментари и отговори.
- Модерация: Преглед и обработка на доклади от потребители, налагане на санкции (strikes) и блокиране на потребители (ban).
- Комуникация: Отговаряне на въпроси и предложения от потребители.

3. Ограничения на системата

- Достъпност: Потребители без регистрация могат само да разглеждат сайта, без възможност за създаване на съдържание или участие в игри.
- Езикова поддръжка: Системата е фокусирана основно върху българскоезично съдържание.
- Модерация: Автоматизираният преглед на съдържанието (AI) е ограничена функционалност, подпомагана от административен контрол.
- Сигурност: Административните функции са строго ограничени чрез защитени маршрути и роли, като всяко нарушение на правилата се следи чрез система за натрупване на предупреждения (3 Strikes Rule).
- Инфраструктура: Проектът използва специфична технологична архитектура (React/TypeScript за фронтенд и Express/Node.js за бекенд), която определя границите на разширяемост при текущата реализация.

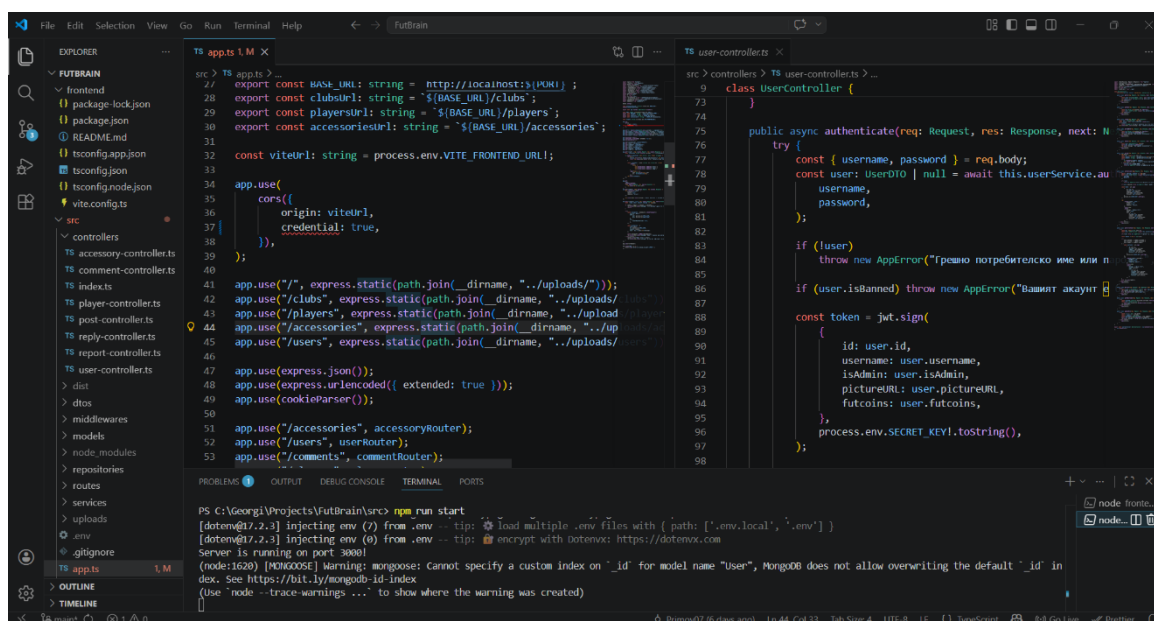


V. Използвани технологии

1. Visual Studio Code

Visual Studio Code представлява водеща среда за разработка, създадена от компанията Microsoft, която съчетава предимствата на лек редактор на изходен код с инструменти за цялостно управление на софтуерни продукти. Архитектурата на софтуера е проектирана така, че да осигурява бързо стартиране и гладка работа дори при обемни проекти. В рамките на FutBrain, този инструмент служи като централно място за съгласуване на клиентската и сървърната логика, като налага висока дисциплина при поддръжката на програмния код. Средата е избрана заради своята вградена и пълна съвместимост с повечето операционни системи, езика TypeScript и средата за изпълнение Node.js. Основните характеристики на средата за разработка са:

- Интегрирани инструменти за откриване и отстраняване на програмни грешки в реално време;
- Вградена поддръжка на системи за контрол на версиите за проследяване на промените;
- Обширен набор от допълнителни модули за работа със съвременни уеб технологии;
- Интегрирана командна конзола за директно управление на системни процеси и скриптове. (фиг.1)



Фигура 1: Работа с VS Code



2. Език за програмиране – TypeScript

TypeScript е строго типизирано надмножество на езика JavaScript, което въвежда възможности за статична типизация към стандартния синтаксис. Технологията е избрана с цел минимизиране на грешките при изпълнение и подобряване на структурата при обработка на сложни данни, като позволява споделяне на дефиниции между клиентската и сървърната част на приложението. В контекста на FutBrain, езикът осигурява високо ниво на сигурност на програмния код и улеснява разработката чрез ясно дефинирани интерфейси. Проектът стриктно следва съвременните спецификации на стандарта ECMAScript, като същевременно налага специфични правила за организация на файловата структура. В сървърната част (бекенд) е възприет стандартът за именуване на файлове чрез малки букви и тирета (kebab-case). За разлика от нея, във фронтенд частта интерфейсите файлове с разширение .tsx се именуват чрез главни букви в началото на всяка дума (PascalCase) – практика, която се е утвърдила като водещ стандарт при работа с библиотеката React. Употребата на тези правила гарантира системност, висока четимост и предвидимост при поддръжката на софтуерния продукт.

3. Backend

3.1 Node.js + Express

Node.js е най-разпространената среда за изпълнение на сървърни приложения, която в съчетание с програмната рамка Express се използва за изграждане на приложно-програмния интерфейс на системата. Тя се използва от много на брой световноизвестни компании за техните уебсайтове като НАСА, Netflix, PayPal и други. Този технологичен слой е избран заради своята способност да обслужва множество едновременни заявки чрез модел на неблокиращ вход и изход (non-blocking I/O), което премахва необходимостта от използване на различни езици за сървър и клиента. Също така той подпомага използването на един и същ език както във фронтенда, така и в бекенда, което улеснява споделянето на логика и модели между тях. Това значително ускорява разработката и намалява възможността за грешки при преноса на данни. Node.js разполага с богата екосистема от готови библиотеки, благодарение на което проектите, базирани на тази среда, могат лесно да интегрират сложни функции, свързани със сигурност връзка с бази от данни например, без да се налага всичко да се пише отначало.

Бекенд рамката Express допълва Node.js, като предоставя инструменти за управление на пътищата и заявките без излишна сложност. Използването на тези технологии във FutBrain позволява бърза обработка на информацията и стабилна връзка



с базата данни. Express предоставя само най-необходимите инструменти за изграждане на уеб сървър, без да налага тежки и сложни структури. Това дава пълна свобода при организирането на папките и файловете във FutBrain. Също така рамката позволява изключително лесно и подредено дефиниране на URL адресите на приложението. Това прави кода лесен за четене и разширяване, когато се появят възможности за развитие. Express използва механизъм от „слоеве“, които могат да проверяват или променят заявките, преди те да достигнат до основната логика. Това са т.нар. „middleware“ функции, или просто „middlewares“.

3.2 База от данни – MongoDB Atlas, Mongoose и Typegoose

MongoDB е нерелационна система за управление на бази данни, основана на документно-ориентиран модел за съхранение на информация във формата BSON (Binary JSON). За разлика от четимия от хората JSON, бинарната структура предлага по-добра производителност, по-богато съхранение и по-богати типове данни (Date, Int32/64). Освен това миграциите към базата от данни са автоматизирани при всяка конфигурационна промяна на обектите, т.е. програмистът няма нужда да се грижи за този процес. MongoDB е предпочетена поради необходимостта от гъвкавост при структурирането на разнообразни спортни и потребителски данни, които невинаги следват строга таблична форма. За FutBrain се предпочита използването на отдалечената (Cloud) база от данни MongoDB Atlas. Тя работи в т.нар. Replica Set (група от сървъри). Това означава, че ако единият от сървърите спре или се повреди, друг веднага поема неговата роля. Също така Atlas предоставят лесна мащабируемост, тъй като спокойно може да бъде увеличен капацитетът на клъстера, на който са разположени колекциите, без да се налага да се преинсталира софтуер или да се купуват нови физически сървъри. Това се случва автоматично и без прекъсване на работата на сайта. Освен това отдалечените доставчици на MongoDB поемат тежката задача по администрирането, като потребителите на базата получават автоматични обновления на сигурността, ежедневни резервни копия и вградена защита срещу атаки. Достъпът до клъстера на базата от данни може да бъде ограничен само до специфични IP адреси, което добавя допълнително ниво на защита за потребителските данни.

3.3. Работа с базата от данни - Mongoose и Typegoose

За взаимодействие с базата данни е внедрена библиотеката Mongoose и нейната TypeScript обвивка Typegoose. Mongoose създава слой между кода на приложението и базата от данни, като дефинира схеми за данните. Тя позволява на разработчика да дефинира строги правила за свойствата на обектите и middleware hooks (междинни куки),



които автоматично се изпълняват преди или след определени действия. Mongoose е най-използваната библиотека за MongoDB и следователно има най-голяма общност. За сайт като FutBrain просто Mongoose не е достатъчен, тъй като, от една страна, тя има слаба интеграция с TypeScript, а от друга, той съдържа голям брой обекти със сложни връзки помежду им. Затова се интегрира обвивката Typegoose. Тя позволява да дефинираме Mongoose модела като клас с декоратори за валидация и така класът служи едновременно и като TypeScript тип, и като Mongoose schema.

3.4. Сигурност - bcrypt

Bcrypt се използва за надеждно хеширане на паролите. Тя осигурява устойчивост срещу brute-force атаки, тъй като използва адаптивен алгоритъм, изискващ значително време за изчисление. Вградената в приложението защита добавя случайно число, наречено сол (salt) към низа с паролата преди хеширане. Това гарантира, че дори двама потребители да имат еднаква парола, техните хешове в базата от данни ще изглеждат напълно различно. Алгоритъмът на хеширане позволява приложението да увеличи интервала на солта с напредването на мощността на компютрите, без да се променя логиката на проекта.

3.5. Авторизация - JSON Web Tokens (JWT)

JWT осигурява защитен механизъм за авторизация чрез цифрови подписи, пазени в бекенда, за да има налице защита срещу преднамерена промяна на потребителските данни. Чрез използването на JWT сървърът не трябва да пази информацията за сесията на потребителя в паметта си. Това прави приложението по-бързо и лесно за мащабиране. Тъй като JWT е стандартен JSON формат, той лесно може да се използва за комуникация между различни услуги (в случая между Node.js + Express бекенда и React фронтенда) без сложни преобразувания.

3.6. Обработка на бисквитките - cookie-parser

Библиотеката cookie-parser е малък, но важен инструмент в бекенд архитектурата на FutBrain, който улеснява работата с данните, съхранявани в брауъра на потребителя. Тя автоматично преобразува сложния низ от символи в заглавата Cookie на HTTP заявката в лесен за използване JavaScript обект. Cookie-parser позволява използването на таен ключ за подписване на бисквитките, което гарантира, че ако потребителят се опита да промени стойността на бисквитката в своя брауър, сървърът веднага ще засече манипулацията и ще я отхвърли като невалидна. Също така библиотеката осигурява



механизми за защита срещу кражба на бисквитки чрез скриптове и CSRF (Cross-Site Request Forgery) атаки.

3.7. Управление на сървърните пътища - CORS

Библиотеката CORS е критичен компонент за сигурността и правилната комуникация между фронтенда и бекенда на приложението. Тя позволява на бекенд сървъра изрично да посочи кои външни домейни имат право да изискват данни. Без тази настройка браузърите блокират заявките от съображения за сигурност. В този контекст CORS действа като филтър, който предпазва API на бекенда от достъп от неавторизирани или потенциално опасни партии.

3.8. Качване и съхранение на файлове - Multer

Библиотеката multer е основен стандарт в работата с файлове в Node.js, който осигурява специализиран middleware за Express. Тя обработва качените файлове на части (streaming - стрийминг), което ѝ позволява да поема големи изображения или документи, без да претоварва оперативната памет на сървъра. Библиотеката предоставя гъвкави опции за това къде и как да се записват файловете. Програмистът може да избере между директно записване на диска (DiskStorage) за постоянно съхранение с автоматично преименуване или съхранение в паметта (MemoryStorage) за бърза обработка преди качване в облачна услуга. Multer позволява лесно дефиниране на ограничения за размера на файловете и проверка на техния тип. Това предпазва сървъра от злонамерени действия като качване на прекомерно големи файлове.

3.9. Управление на изходяща комуникация чрез ел.поща – Nodemailer

За връзката между потребителите и администрацията чрез формална комуникация се грижи библиотеката Nodemailer. Тя поддържа почти всички съвременни методи за изпращане на имейли – от стандартни SMTP сървъри и услуги като Gmail и Outlook, до специализирани платформи като Amazon SES. Nodemailer позволява лесно създаване на имейл с форматиран HTML текст, вградени изображения и прикачени файлове. Библиотеката предлага вградена поддръжка за криптиране (SSL/TLS) и автентикация, което гарантира, че данните на потребителите и съдържанието на имейлите са защитени по време на преноса. Освен това тя се справя отлично с технически детайли като кодиране на символи, което предотвратява проблеми с визуализацията на текстовете на кирилица.



3.10. Внедряване на функции с ИИ за генериране на съдържание - google/generative-ai/gemini-2.5-flash-lite

За модерация на въведеното от потребителя съдържание се използва моделът Gemini 2.5 Flash-Lite. Той автоматизира част от административната работа, позволявайки на разработчика да се фокусира върху развитието на функционалностите, вместо върху постоянна ръчна проверка на публикации и коментари. Използването на Gemini 2.5 Flash-Lite носи значителни технологични предимства, тъй като този модел е специално оптимизиран за задачи, изискващи баланс между скорост и прецизност, а генерално Gemini води в статистиките по модерация на съдържанието (вж. „Използвана литература“). Първият от плюсовете е ниската латентност. Като част от „Lite“ серията моделът е проектиран за мигновена обработка на входящи текстове и коментари. Това позволява на FutBrain да извършва модерация в реално време, като блокира неподходящо съдържание още в момента на опита за публикуване. Gemini 2.5 Flash-Lite е оптимизиран за задачи с висок капацитет при минимални разходи на токен (единица информация, която изкуственият интелект обработва). Това го прави правилното решение за мащабируеми платформи като FutBrain, където обемът от потребителско съдържание може да нарасне бързо, без това да доведе до прекомерни разходи за поддръжка на ИИ.

3.11. Конфигурация на средата - dotenv

Библиотеката dotenv е фундаментален инструмент за сигурността и организацията на FutBrain. Нейната основна задача е да зарежда променливи на средата (environment variables) от специален файл .env – пароли за базата от данни (MongoDB URI), секретни ключове за JWT или API ключове за Gemini извън програмния код. Складирането на тази чувствителна информация в този файл е ключово за нейната защита и така споделянето на кода в платформи като GitHub става безопасно. С dotenv лесно могат да се променят конфигурациите на приложението в зависимост от това дали работи локално (в процес на разработка) или е качено на сървър (режим на работа). Чрез събирането на глобалните настройки на приложението на едно място се подобрява четимостта на кода и се улеснява разбирането му, като му се спестяват дългите низове.

4. Frontend

Изграждането на съвременен и динамичен потребителски интерфейс за проекта FutBrain изисква съчетаването на инструменти, които едновременно осигуряват функционалност и висока скорост на работа.



4.1. Библиотека за потребителски интерфейс - React

React е най-популярната библиотека за изграждане на графични потребителски интерфейси. Нейната архитектура е създадена, за да реши проблемите със скоростта и сложността при управлението на големи уебприложения. Това е така, защото в React интерфейсът не е един монолитен блок, а съвкупност от малки, независими и преизползваеми части, наречени компоненти. Този вид архитектура се нарича „компонентно-базирана архитектура“. Библиотеката осигурява рутиране между различните уебстраници. Технологиата, която придава голямата скорост на работа на React, се нарича виртуален DOM (документно-ориентиран модел). Вместо да обновява цялата страница при всяка промяна, React променя само конкретния елемент. Това осигурява плавно потребителско изживяване. Поради популярността ѝ за библиотеката съществуват готови решения за почти всичко - от анимации до сложни форми и графики.

4.2. Инструмент за изграждане и разработка - Vite

За разлика от по-старите инструменти Vite не компилира целия проект, преди да започне работа. Той използва модерна технология – Native ESM (ECMAScript Modules), при която браузърите четат и свързват различни файлове на приложението. В миналото браузърите не са разбирали команди като `import` и `export` директно, а сега те имат вградена поддръжка на ESM. Това означава, че се зарежда само кодът, който е необходим за текущия изглед, а няма нужда сървърът да генерира цялата информация на уебсайта, по-голямата част от която е ненужна. Също така, когато се променя кодът, Vite отразява промяната в браузъра веднага, без да губи състоянието на приложението. Когато FutBrain е готов за качване в продукцията, Vite използва високопроизводителния инструмент Rollup, за да минимизира и оптимизира кода на уебсайта. Резултатът са изключително леки и бързи файлове, които потребителите изтеглят за секунди.

4.3. Стилизиране на компонентите и адаптивен дизайн - CSS

Чрез съхранението на стиловете на различни визуални компоненти в отделени CSS файлове имаме разделение на отговорностите (Separation of Concerns) между страниците и самите компоненти в тях. Държейки информацията и външния вид напълно отделени, можем да променим цветовете, шрифтовете и цялостното оформление на страницата, редактирайки само един файл. Освен това CSS дава възможност за адаптивен дизайн спрямо устройството на клиента чрез пренареждане, скриване или свиване на елементите. Това във FutBrain е реализирано чрез Media Queries и Flexbox. Уебприложението ще изглежда добре както на голям монитор, така и на мобилно устройство. Чистият и добре структуриран дизайн помага на потребителите да се



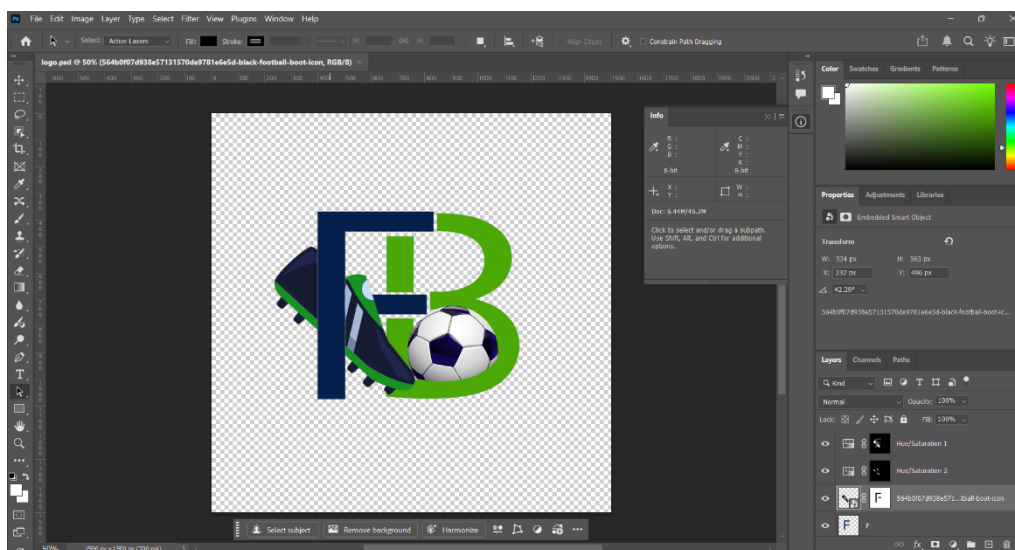
ориентиран по-лесно в сайта. CSS е изключително лек език, който се кешира от браузъра при първото посещение.

4.4. Икони за модерен изглед - Font Awesome

Font Awesome е стандартизирана библиотека, предоставяща набор от мащабируеми векторни икони за интегриране в потребителския интерфейс. Технологията е предпочетена пред ръчното създаване и зареждане на отделни графични файлове, с цел уеднаквяване на визуалния език и намаляване на мрежовото натоварване на уеб приложението.

5. Лого на FutBrain – Adobe Photoshop

Adobe Photoshop е специализиран софтуер за обработка на растрерна графика, използван в проекта за изработка на конкретни визуални активи, като логото на системата. Програмата е избрана поради наличието на прецизен контрол върху пикселите, цветовете профили и слоевете, което гарантира оптимална подготовка на графичните елементи за интегриране в уеб среда. (фиг. 2).



Фигура 2: Изработка на логото в Adobe Photoshop



VI. Системна архитектура

Приложението използва най-общото архитектурно разделение на файловата си структура на backend (src/) и frontend (frontend/).

1. Backend архитектура (src/) (фиг. 4)

1.1. Models (models/)

Този слой дефинира схемите на данните чрез Mongoose. Той установява задължителната структура на базата от данни и правилата за валидация на основните обекти в системата, като потребителски профили, публикации и коментари.

1.2. Repositories (repositories/)

Репозиторитата представляват специализиран слой за директна комуникация с базата данни MongoDB. Те капсулират CRUD операциите (създаване, четене, актуализация и изтриване) и исигуряват техническа изолация на базата данни от бизнес логиката на приложението.

1.3. Services (services/)

В този слой е интегрирана цялостната бизнес логика на проекта. Тук се извършват критични процеси като автоматизирана AI модерация чрез Gemini и софтуерно изчисляване на виртуалната валута на сайта (futcoins).

1.4. Routes (routes/)

Този модул дефинира публичните крайни точки (endpoints) на приложния програмен интерфейс (API). Той координира връзката между специфичните URL адреси и съответните методи в контролерите.

1.5. Middlewares (middlewares/)

Междинният софтуер изпълнява спомагателни функции, свързани със сигурността и системната конфигурация. В случая той извършва автентикацията на потребителите чрез проверка на JWT токени и управлява обработката на файлови потоци при качване на съдържание чрез Multer.

1.6. DTO-та (dtos/)

Този слой дефинира обектите за трансфер на данни, които специфицират формата на информацията, пренасяна между клиента и сървъра. Обектите служат за стриктна валидация на входящите данни при заявки за създаване или актуализация. Също така те Предотвратяват излагането на чувствителна вътрешна информация, като дефинират точно кои полета от моделите могат да бъдат изпращани към фронтенда.



1.7. Node Modules (node_modules/)

Директорията съдържа всички външни библиотеки и зависимости, необходими за функционирането на проекта, инсталирани чрез пакетен мениджър (Node Package Manager - npm). Тя автоматично се генерира и не подлежи на ръчна модификация или включване в системата за контрол на версиите (Git). Папката осигурява локалната среда с изпълним код за инструменти като Express, Mongoose и TypeScript компилатора.

1.8. Distribution (dist/)

Тази папка съдържа крайния, компилиран и оптимизиран JavaScript код, генериран от TypeScript източниците в src/ (кореновата директория на местния tsconfig.json файл). Процесът на компилация трансформира модерния TypeScript синтаксис в стандартизиран JavaScript, гарантирайки съвместимост и висока производителност на сървъра. Тук се съхранява версията на приложението, която се изпълнява в продукционна среда.

1.9. Uploads (uploads/)

Директорията изпълнява ролята на локално хранилище за динамично съдържание, генерирано от потребителите на FutBrain. В нея се организират ресурси като потребителски аватари, прикачени изображения към публикации и снимки на дигитални аксесоари, футболни клубове и футболни играчи. Директорията се конфигурира като статичен път в Express, което позволява на фронтенда да осъществява директен достъп до файловете чрез уникални URL адреси. Съдържанието ѝ се изключва от системата за контрол на версиите, за да се избегне неоправдано увеличаване на размера на хранилището с клиентски данни.

1.10. App (/app.ts)

Този файл дефинира инстанцията на Express приложението и централизира интеграцията на всички глобални функционалности. В него се инициират и подреждат междинните софтуери (middlewares) като cors, express.json(), cookie-parser и обслужването на статични файлове от директорията /uploads. Файлът изпълнява ролята на свързващо звено, което насочва входящия трафик към съответните маршрути (routes) и дефинира глобалните стратегии за обработка на грешки.

2. Frontend архитектура (frontend/) (фиг. 3)

2.1. Public (public/)

Тази директория съдържа статични ресурси, които се обслужват директно от браузъра без намесата на системата за компилация на React. В случая тя включва логото



на FutBrain, което се визуализира в навигационното меню на админ панела и потребителските страници. Ресурсите в директорията са достъпни чрез абсолютни пътища спрямо корена на домейна.

2.2. Pages (src/pages/)

Страниците са основните компоненти, които изграждат потребителския интерфейс в React. Всяка страница комуникира с бекенда чрез `fetch` заявки към бекенда и организира визуализацията на получените данни за крайния потребител.

2.3. Layouts (src/layouts/)

Слоят на оформлението (layouts) дефинира общата визуална рамка на приложението, която се споделя между множество страници. Той капсулира повтарящи се структурни елементи като навигационни ленти, хедъри (Header) и футъри (Footers). Слоят осигурява консистентност на интерфейса и позволява лесна промяна на общия дизайн, без да се засяга индивидуалната логика на отделните страници.

2.4. Styles (src/styles)

Тази папка съдържа глобалните CSS или дефиниции, които определят естетиката и типографията на целия проект. В нея се дефинират системни променливи (като цветови палитри, размери на шрифтове и отстояния), което гарантира визуална консистенция между отделните компоненти.

2.4. Auth (src/auth/)

Това е механизъм за глобално управление на състоянието на автентикация в приложението. Той осигурява необходимите данни за идентификация на потребителя и служи за прилагане на ограничителни филтри при достъп до защитени ресурси (ProtectedRoute).

2.5. DTO-та (src/dtos/)

DTO-тата във фронтенда специфицират точните интерфейси и типове данни, прехвърляни от сървъра. Те гарантират строга типова безопасност и съгласуваност между структурата на данните в бекенда и тяхното използване в потребителския интерфейс.

2.6. App (src/App.tsx)

Този файл дефинира основната структура на потребителския интерфейс и служи като централен разпределител на логиката в клиентската част. В него се конфигурира декларативната маршрутизация, която определя коя страница да се визуализира спрямо текущия URL адрес. Тук се интегрират глобалните доставчици на състояние (Providers),



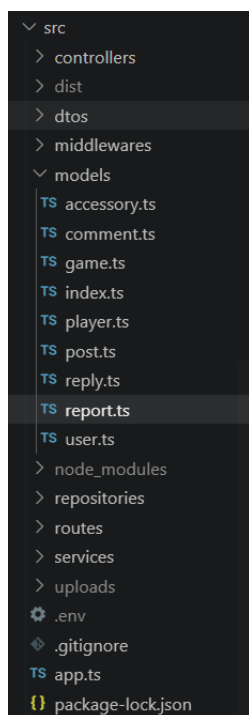
като например AuthContext, осигурявайки достъп до споделени данни за всички подкомпоненти. Файлът организира йерархията на компонентите, като обвива маршрутите в споделени оформлени (Layouts), за да гарантира консистентно визуално присъствие на навигационните елементи.

2.7. Main (src/main.tsx)

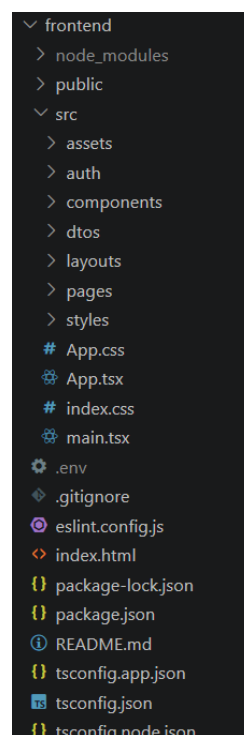
Този файл изпълнява ролята на свързващо звено между React приложението и реалния DOM на брауъра. В него се извършва рендирането на кореновия компонент (App.tsx) в конкретен HTML елемент (обикновено `<div id="root"></div>`), дефиниран в основния index.html (src/index.html). Тук се импортират глобалните стилове от frontend/src/styles, за да се гарантира, че те ще бъдат приложени върху цялото приложение още при неговото зареждане.

2.8. Node modules (/node_modules)

Тази папка съдържа компилирания код на всички зависимости, дефинирани във файла package.json на фронтенда, като React и Vite. Тя служи като локална база от ресурси, от която системата за компилация черпи необходимите модули за изграждане на потребителския интерфейс. Поради огромния си обем и автоматизирания начин на генериране, директорията задължително се изключва от системата за контрол на версиите чрез .gitignore.



Фигура 4:
Архитектура на
бекенда



Фигура 3:
Архитектура на
фронтенда



VI. Описание на системата

1. Архитектура на базата от данни (фиг. 5)

1.1. User (Потребител)

Това е централният обект в системата, който управлява идентичността, сигурността и социалната активност. Той интегрира автоматизирано хеширане на пароли чрез bcrypt и управление на потребителски статус (администратор, наказания, валута futcoins). Поддържа масиви от референции към публикации, коментари, отговори, харесвания (публикации, коментари и отговори) и закупени аксесоари.

1.2. Accessory (Аксесоар)

Обектът представлява виртуален предмет (топка, емблема или обувки), достъпен в рамките на магазина на платформата. Използва енумерация (изброимост) за категоризация и автоматично генерира пътища към снимки на базата на уникалния идентификатор на обекта. Налага строги правила за положителна целочислена стойност на цената и задължителен краен срок на валидност.

1.3. Comment (Коментар)

Този обект служи като първично ниво на социална интеракция под публикациите в платформата. Съдържа текстово съдържание с минимална дължина от 10 знака и поддържа прикачване на множество снимки. Осъществява двупосочна връзка между конкретен потребител и публикация, като същевременно агрегира масив от отговори (replies).

1.4. Post (Публикация)

Основният информационен субект, около който се изгражда потребителското съдържание. Налага валидация за минимална дължина на заглавието и съдържанието, като съхранява дата на публикуване и медия по избор.

1.5. Reply (Отговор на коментар)

Представлява второ ниво на комуникация, вложено в рамките на коментарите. Структурата му е идентична с тази на коментарите, но е строго дефиниран като подчинен елемент на конкретен такъв. Реферира едновременно към родителския коментар и към автора на отговора.

1.6. Player (Футболен играч)

Моделът описва професионални футболни атлети, интегрирани в екосистемата на FutBrain. Автоматизира логиката за асоцииране на клубни емблеми и изображения на играчите чрез предварително дефинирани системни пътища. Поддържа списък от



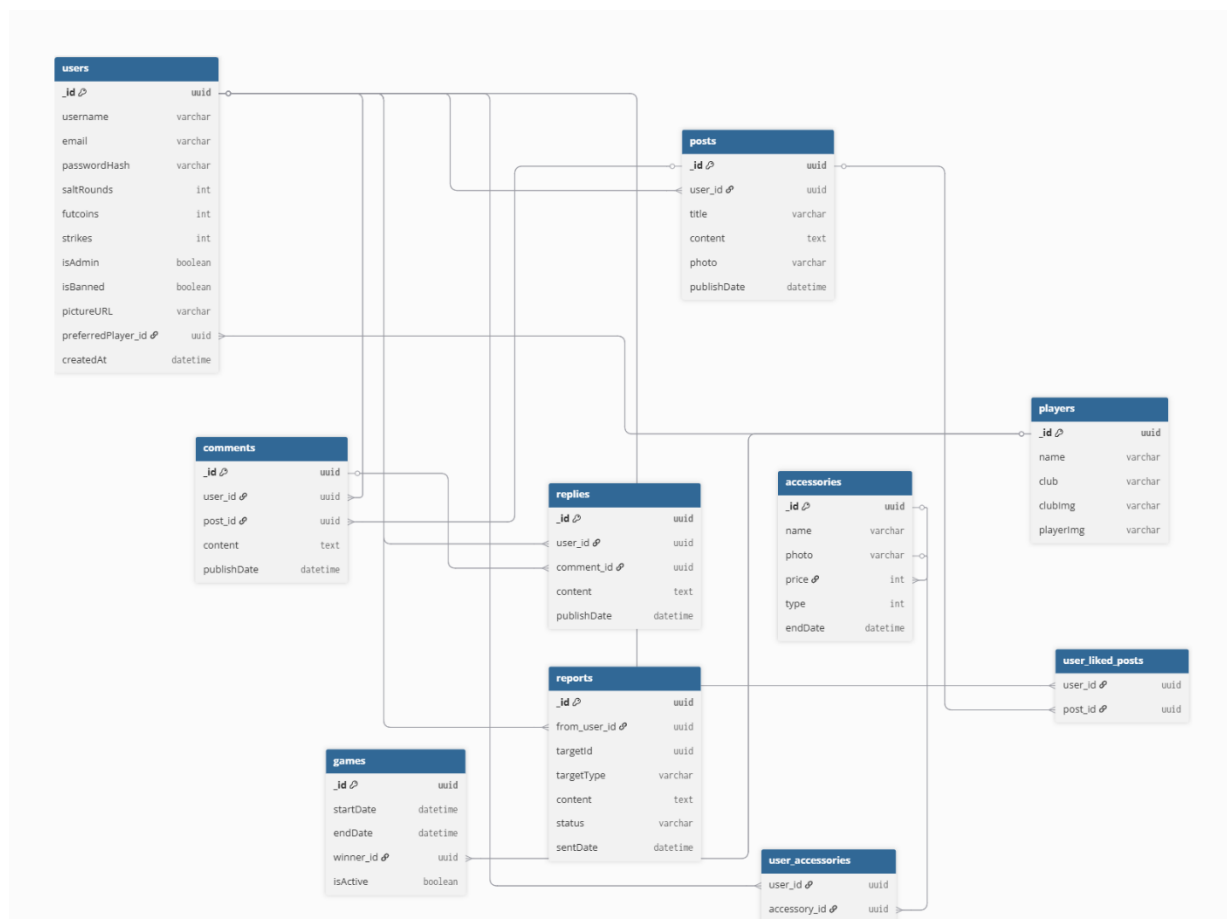
потребители, които са посочили този атлет като свой избран за съответния игрови модул футболист.

1.7. Game (Игра)

Обектът капсулира данни за игровите сесии в уебплатформата. Следи времеви прозорец на активност (начална и крайна дата) и управлява текущия статус чрез флага isActive. Реферира към обекта Player, който е определен като победител в конкретната сесия.

1.8. Report (Доклад)

Това е модерационен инструмент за проследяване на нарушения в платформата. Използва полиморфна логика чрез полетата targetType и targetId, за да идентифицира докладваното съдържание (Post, Reply или Comment). Поддържа жизнения цикъл на обработка на доклада чрез състоянията Pending, Resolved и Rejected.



Фигура 5: Архитектура на базата от данни



2. Алгоритми

2.1. Алгоритъм за сигурност и криптографска защита

Процесът по автентикация се основава на многопластов алгоритъм за защита на потребителските данни. При регистрация или промяна на парола, системата автоматично генерира динамичен брой итерации („salt rounds“) чрез библиотеката `crypto`. След това паролата се обработва чрез `bcrypt` хешираща функция. Този алгоритъм гарантира, че дори при пробив в базата данни, оригиналните пароли остават неразчетими. Преди запис се прилага регулярен израз (Regex) за проверка на комплексността на паролата (минимум 8 символа, цифра и специален знак).

2.2. Алгоритъм за автоматизирана AI модерация

Този алгоритъм е интегриран в `Services` слой и използва `Gemini API` за анализ на генерираното от потребителите съдържание в реално време. Последователността от стъпки е следната: текстът на публикация, коментар или отговор се изпраща към изкуствения интелект. След това моделът анализира съдържанието за токсичност, нецензурно, расистко, хомофобско или политическо съдържание. Ако се открие нарушение, алгоритъмът връща грешка на потребителя, известяваща го за неподходящия контекст на публикацията, коментара или отговора, и му се дава възможност да смени тона на комуникация.

2.3. Алгоритъм за управление на виртуалната икономика (Futcoins)

Логиката за управление на транзакциите във `FutBrain` следи за баланса между активност и възнаграждение. Потребителите печелят `futcoins` чрез участие в предназначения игрови модул „Най-добър играч на седмицата“. При закупуване на аксесоар, алгоритъмът извършва проверка за наличност, дебитира сметката на потребителя и добавя референция към аксесоара в неговия масив `accessories`. Процесът е защитен чрез транзакционна логика, за да се предотвратят грешки при дублиране на покупки.

2.4. Логика за динамично управление на ресурси

Системата прилага алгоритъм за автоматично генериране на `URL` адреси за медийно съдържание. Чрез използването на уникалните идентификатори (`UUID`) на обектите и предварително дефинирани константи за базови адреси (`BASE_URL`), алгоритъмът конструира пътища към снимки на играчи, клубове и аксесоари без необходимост от ръчно въвеждане на данни от администратора.



3. Документация на кода

3.1. Декоратори и Hooks (@pre "save")

В моделите на Typegoose се използват @pre декоратори, които действат като „куки“ преди запис в базата данни. Те автоматизират подготовката на данните. В архитектурата на FutBrain куките служат за:

- Автоматично конвертиране на _id в string формат за полето id на всички обекти без user;
- Хеширане на паролата на потребителя;
- Генериране на системни аватари по подразбиране за нови потребители и сол за алгоритъма по хеширане;
- Актуализиране на снимките на клубовете при промяна на принадлежността на играча.

```
@pre<User>("save", async function () {
  if (this.isModified("passwordHash")) {
    const regex : RegExp =
      /^(?=.*[A-Za-z])(?=.*\d)(?=.*[!@#%&*])[A-Za-z\d!@#%&*]{8,}$/;
    if (regex.test(this.passwordHash)) {
      this.saltRounds = randomInt(1, 11);
      this.passwordHash = await bcrypt.hash(this.passwordHash, this.saltRounds);
    }
    else throw new AppError("Паролата трябва да бъде дълга поне 8 символа, да съдържа поне една буква, цифра и специален символ!", 400);
  }
  if (this.isModified("strikes") && this.strikes == 3) this.isBanned = true;
  if (this.isNew && !this.pictureURL) {
    this.pictureURL = `${BASE_URL}/user.png`;
  }
})
```

Фигура 6: @pre "save" кука на обекта User

3.2. Клас за централизирано управление на грешки (AppError)

Този клас разширява стандартния обект Error в TypeScript, добавяйки полета за HTTP статус код и съобщение, пригодно за крайния потребител. Класът позволява на разработчика да прекъсва изпълнението на логиката по контролиран начин, което

активира глобалния error-handler и изпраща форматиран JSON отговор към фронтенда (фиг. 7).

```
public async authenticate(req: Request, res: Response, next: NextFunction) {
  try {
    const { username, password } = req.body;
    const user: UserDTO | null = await this.userService.authenticate(
      username,
      password,
    );

    if (!user)
      throw new AppError("Грешно потребителско име или парола!", 401);

    if (user.isBanned) throw new AppError("Вашият акаунт е блокиран!", 403);
  }
}
```

Фигура 7: Хвърляне на AppError при автентикация

3.3. Middleware за JWT автентикация

Това е ключова функция, разположена в middlewares/, която прихваща всяка защитена заявка. Тя извлича JSON Web Token от заглавните части (headers) или бисквитките, декодира го и верифицира самоличността на потребителя. При успешно съвпадение, обектът на потребителя се закача към заявката (req.user), правейки го достъпен за следващите нива на бизнес логиката.

3.4. Метод comparePasswords

Дефиниран в класа User, този асинхронен метод капсулира логиката по сравняване на въведената от потребителя парола в чист текст с криптирания хеш в базата данни. Използването на този метод в контролерите гарантира, че бизнес логиката не работи директно с хеширащи алгоритми, спазвайки принципа на капсулация.



VII. Ръководство за потребителя

Проектът е създаден с фокус върху лесната употреба, като предоставя интуитивен интерфейс, ясна навигация и минимална нужда от технически познания. Всички компоненти и функционалности са проектирани така, че потребителят бързо да се ориентира и да извърши желаното действие с минимални усилия.

1. Инструкции за работа със системата

1.1. Регистрация и сигурност на достъпа

За пълноценно използване на интерактивните модули на FutBrain, потребителят трябва да премине през процес на регистрация. Той включва:

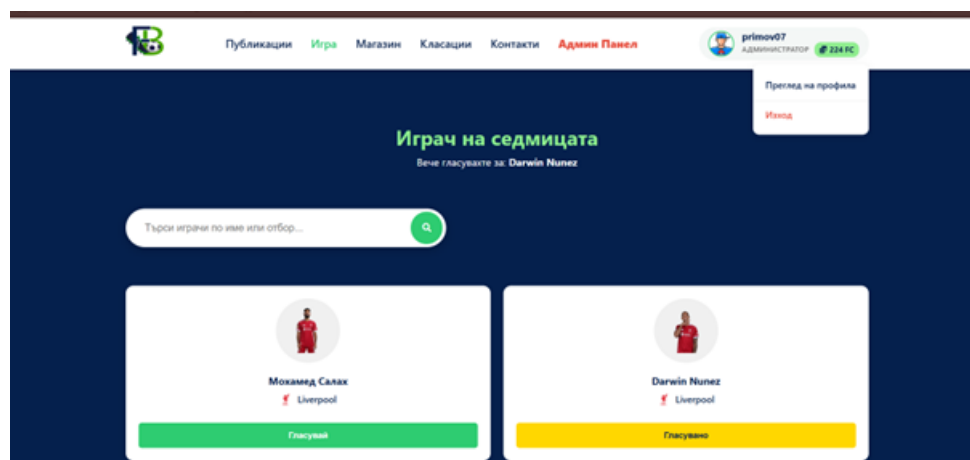
- **Идентификация:** Въвеждане на валиден имейл адрес и уникално потребителско име (минимум 7 символа).
- **Криптографска защита:** Създаване на парола, която системата автоматично хешира. Валидацията изисква дължина от поне 8 знака, наличие на цифра и специален символ.
- **Първоначална настройка:** Системата автоматично генерира базов профил с начален капитал от Futcoins и системен аватар, съхраняван в директорията uploads.

1.2. Навигация и потребителски интерфейс

Интерфейсът е разделен на логически зони за максимално удобство: От една страна, имаме горна навигация, която осигурява бърз достъп до профила, магазина за аксесоари, активната игрова сесия, класациите и страницата с контакти. От друга страна, имаме централния поток (Feed-a), където потребителят може да чете, харесва и коментира съдържание, да извършва комуникация с администрацията под формата на



доклади или email, да се възползва от натрупаното си FutCoins състояние или да гласува за своите любимци в „Най-добър играч на седмицата“ (фиг. 8).

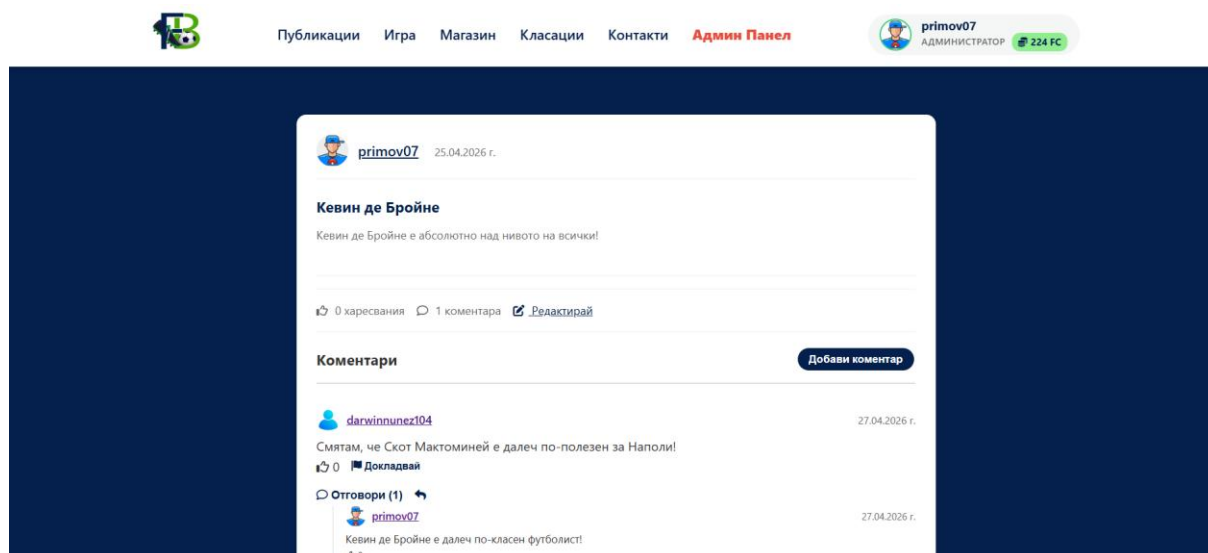


Фигура 8: След гласуване в "Най-добър играч на седмицата"

1.3. Социална интеракция и модерация

Потребителите могат активно да участват в общността чрез:

- Публикации и коментари: Създаване на съдържание (минимум 5 знака за заглавие и 10 за текст), към което могат да се прикачват изображения.
- Система за доклади (Reporting): Всеки потребител може да сигнализира за неуместно съдържание, което той смята, че не е било засечено от AI модерацията.



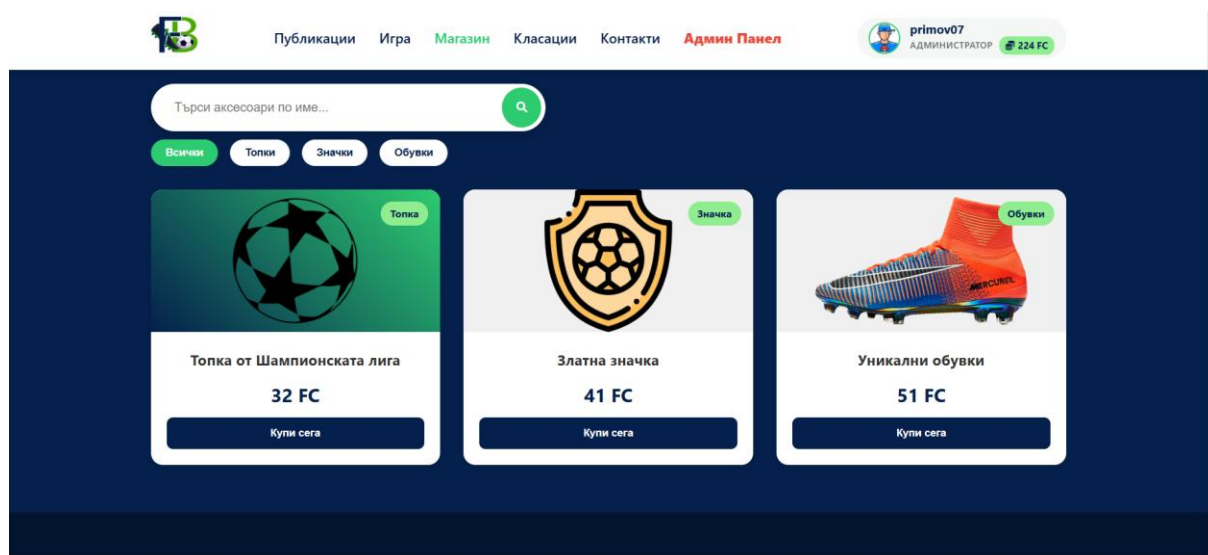
Фигура 9: Преглед на публикация с коментари и отговори

1.4. Магазин за аксесоари и инвентар:

Виртуалният магазин позволява покупка на дигитални предмети:



- Избор на аксесоари (топки, емблеми, обувки) чрез филтриране по тип.
- Автоматично изчисляване на транзакцията и актуализиране на баланса от Futcoins. Закупените предмети се визуализират в личния профил на потребителя. (фиг. 10)



Фигура 10: Изглед на магазина на FutBrain

1.5. Игрови модул (Game)

Системата визуализира текущия победител и активността на сесията, като награждава участниците при успешно приключване на събитието.



VIII. Авторски права

Настоящата дипломна работа и свързаният с нея софтуерен код са защитени с авторски права Георги Примов ©. Всички права запазени. Библиотеките, използвани в проекта, са или „отворен код“, или се разпространяват с MIT лиценз.



IX. Заключение

В настоящия дипломен проект беше разработена иновативна уеб платформа за футболна общност и социална интеракция – **FutBrain**. Чрез прецизно планиране и използване на най-съвременни технологии, проектът постигна своята фундаментална цел: създаването на бърза, сигурна и мащабируема екосистема, която обединява спортните фенове чрез динамично съдържание, виртуална икономика и усъвършенствана AI модерация.

В началния етап на разработката бе проведено детайлно проучване на съвременните социални мрежи и спортни портали, с цел идентифициране на ниши за иновации. Логиката и архитектурата на приложението бяха успешно реализирани чрез модерния технологичен стек React (Vite) за фронтенда и Node.js/Express с TypeScript за бекенда. Използването на MongoDB в комбинация с Typegoose осигури гъвкава, но строго типизирана структура на данните, гарантираща висока производителност при обработка на сложни релации между потребители, публикации и коментари.

Приложението предлага богат набор от функционалности, включително сигурна автентикация чрез JWT и bcrypt, динамичен фийд от съдържание и уникална система за управление на виртуална валута (Futcoins) и дигитални аксесоари. Ключов иновативен елемент е интегрирането на Gemini API за автоматизирана AI модерация, която в реално време следи за токсичност и автоматично прилага система от наказания (strikes), осигурявайки безопасна среда за потребителите.

Разработката бе съпътствана от сериозни предизвикателства, като изграждането на полиморфна система за доклади, управлението на сложни асинхронни процеси при обработка на файлови потоци чрез Multer и поддържането на консистентност между различните нива на архитектурата (DTO-та, Services, Repositories). Благодарение на прилагането на чист програмен код и софтуерни инженерни практики, тези трудности бяха успешно преодолені.

Въпреки че проектът изпълнява всички поставени цели, съществуват значителни възможности за бъдещо развитие, като:

- **Разработка на мобилно приложение** – създаването на native приложение (напр. чрез React Native) ще позволи на потребителите да следят футболните новини и своите активности чрез известия в реално време;



- **Интегриране на реални футболни данни** – внедряване на външни API-та за резултати на живо, статистики и новини от световните първенства;
- **Разширяване на AI функционалностите** – използване на изкуствен интелект не само за модерация, но и за персонализирани препоръки на съдържание и прогнозиране на резултати;
- **Внедряване на WebSockets** – добавяне на чат функционалност в реално време между потребителите и живи дискусии по време на мачове;
- **Разширяване на виртуалната екосистема** – добавяне на пазар за търговия между потребители (marketplace) и по-сложни игрови механики за печелене на Futcoins;
- **Усъвършенстване на UI/UX дизайна** – добавяне на сложни микро-анимации и „dark mode“ функционалност, които да направят потребителското изживяване още по-грабващо и модерно.

Създадената платформа **FutBrain** предлага модерно и сигурно решение за дигитализация на футболната страст. Избраните архитектурни решения осигуряват стабилност и възможност за лесно надграждане. Приложението има потенциал да се превърне в значима социална среда, насърчаваща спортната култура, отговорното поведение онлайн и сигурната дигитална комуникация.



Х. Използвана литература

[1] Martin, R. C., Clean Code: A Handbook of Agile Software Craftsmanship, 2008. ISBN: 978-0132350884.

[2] Node.js Foundation, „Node.js Documentation,“ 2026. [Онлайн]. Available: <https://nodejs.org/en/docs/>

[3] Meta Platforms Inc., „React Documentation,“ 2026. [Онлайн]. Available: <https://react.dev/>

[4] Microsoft Corp., „TypeScript Handbook,“ 2026. [Онлайн]. Available: <https://www.typescriptlang.org/docs/>

[5] Automattic Inc., „Mongoose ODM Documentation,“ 2026. [Онлайн]. Available: <https://mongoosejs.com/docs/>

[6] Typegoose Team, „Typegoose Documentation,“ 2026. [Онлайн]. Available: <https://typegoose.github.io/typegoose/>

[7] Expressjs.com, „Express Framework Documentation,“ 2026. [Онлайн]. Available: <https://expressjs.com/>

[8] Google LLC, „Gemini API Documentation,“ 2026. [Онлайн]. Available: <https://ai.google.dev/docs>

[9] Mozilla Foundation, „MDN Web Docs,“ 2026. [Онлайн]. Available: <https://developer.mozilla.org/>